

6. Импорт данных

Задача: создать структуру проекта, разработать класс для работы с базой данных и выполнить импорт данных соответственно варианту. Убедиться, что импорт прошел успешно.

6.1. Создание структуры проекта

1. Создать директорию проекта C:\shop_app
2. Поместить в нее директорию с вариантами заданий C:\shop_app\imports
3. Создать пустые файлы main.py, db.py, import_module.py
 - main.py – основной код программы
 - db.py – модуль для работы с базой данных
 - import_module.py – модуль для импорта данных варианта

Структура после этого выглядит так:

```
shop_app/
├── imports/
│   ├── 01_ГаджетМаркет/
│   │   ├── products.xlsx
│   │   ├── users.xlsx
│   │   ├── orders.xlsx
│   │   └── pickup_points.xlsx
│   ├── 02_Книголенд/
│   └── 03_СофтЛайн/
├── main.py          (пустой)
├── import_module.py (пустой)
└── db.py            (пустой)
```

6.2. Создание класса для работы с базой данных.

Откройте db.py в VS Code, введите следующий код и изучите его:

```
import psycopg2
from psycopg2 import Error

class Database:
    """
    Класс для работы с базой данных PostgreSQL
    Обеспечивает подключение и выполнение SQL-запросов
    """

    def __init__(self, host='localhost', database='shop_db', user='postgres', password='123'):
        """
        Инициализация параметров подключения к БД

        Параметры:
        host - адрес сервера БД (localhost для локальной БД)
        database - имя базы данных
        user - имя пользователя PostgreSQL
        password - пароль (ВАЖНО: замените на свой)
        """
        self.host = host
```

```

        self.database = database
        self.user = user
        self.password = password
        self.conn = None

def connect(self):
    """
    Установка соединения с базой данных

    Возвращает:
        True - соединение установлено успешно
        False - ошибка подключения
    """
    try:
        self.conn = psycopg2.connect(
            host=self.host,
            database=self.database,
            user=self.user,
            password=self.password
        )
        return True
    except Error as e:
        print(f"Ошибка подключения к БД: {e}")
        return False

def close(self):
    """Закрытие соединения с БД"""
    if self.conn:
        self.conn.close()

def execute_query(self, query, params=None):
    """
    Выполнение SELECT запроса

    Параметры:
        query - SQL запрос с placeholder %s
        params - кортеж значений для подстановки

    Возвращает:
        Список кортежей с результатами или None при ошибке
    """
    if not self.conn:
        return None

    try:
        cur = self.conn.cursor()
        if params:
            cur.execute(query, params)
        else:
            cur.execute(query)
        result = cur.fetchall()
        cur.close()
        return result
    except Error as e:
        print(f"Ошибка выполнения запроса: {e}")
        return None

def execute_insert(self, query, params=None):
    """
    Выполнение INSERT/UPDATE запроса с commit

    Параметры:
        query - SQL запрос
        params - кортеж значений

    Возвращает:
        True - успешно
        False - ошибка
    """

```

```

if not self.conn:
    return None

try:
    cur = self.conn.cursor()
    if params:
        cur.execute(query, params)
    else:
        cur.execute(query)
    self.conn.commit()
    cur.close()
    return True
except Error as e:
    self.conn.rollback()
    print(f"Ошибка вставки данных: {e}")
    return False

def get_id_by_name(self, table, name_column, name_value):
    """
    Универсальный метод получения ID записи по имени

    Параметры:
        table - имя таблицы
        name_column - имя колонки для поиска
        name_value - искомое значение

    Возвращает:
        ID записи или None

    Пример:
        get_id_by_name('categories', 'name', 'Электроника')
        вернёт ID категории "Электроника"
    """
    query = f"SELECT id FROM {table} WHERE {name_column} = %s"
    result = self.execute_query(query, (name_value,))
    return result[0][0] if result else None

```

Отредактируйте код, измените параметры подключения к базе данных (метод `__init__`, впишите свои данные для подключения, логин, пароль, название БД). Сохраните файл `db.py`

6.3. Реализация импорта данных варианта

Откройте `import_module.py`, вставьте следующий код и изучите его:

```

import pandas as pd
from pathlib import Path
from db import Database

class DataImporter:
    # Класс для импорта данных из Excel-файлов в базу данных.
    def __init__(self, db, folder_name):
        self.db = db
        self.folder_name = folder_name
        self.base_path = Path('imports') / folder_name

    # Импорт категорий (универсальные значения из столбца 'Категория')
    def import_categories(self, df_products):
        categories = df_products['Категория'].unique()

        for category in categories:
            query = "INSERT INTO categories (name) VALUES (%s) ON CONFLICT (name) DO NOTHING"
            self.db.execute_insert(query, (category,))

```

```

        print(f" Категорий: {len(categories)}")
        return True

# Импорт производителей (столбец 'Производитель')
def import_manufacturers(self, df_products):
    manufacturers = df_products['Производитель'].dropna().unique()

    for manufacturer in manufacturers:
        query = "INSERT INTO manufacturers (name) VALUES (%s) ON CONFLICT (name) DO NOTHING"
        self.db.execute_insert(query, (manufacturer,))

    print(f" Производителей: {len(manufacturers)}")
    return True

# Импорт поставщиков (столбец 'Поставщик')
def import_suppliers(self, df_products):
    suppliers = df_products['Поставщик'].dropna().unique()

    for supplier in suppliers:
        query = "INSERT INTO suppliers (name) VALUES (%s) ON CONFLICT (name) DO NOTHING"
        self.db.execute_insert(query, (supplier,))

    print(f" Поставщиков: {len(suppliers)}")
    return True

# Импорт товаров, использует внешний ключи по именам/названиям
def import_products(self, df_products):
    count = 0

    for _, row in df_products.iterrows():
        category_id = self.db.get_id_by_name('categories', 'name', row['Категория'])

        manufacturer_name = row['Производитель']
        manufacturer_id = None
        if pd.notna(manufacturer_name):
            manufacturer_id = self.db.get_id_by_name('manufacturers', 'name', manufacturer_name)

        supplier_name = row['Поставщик']
        supplier_id = None
        if pd.notna(supplier_name):
            supplier_id = self.db.get_id_by_name('suppliers', 'name', supplier_name)

        query = """
            INSERT INTO products (
                sku, name, description, category_id, manufacturer_id,
                supplier_id, unit, price, discount_percent,
                stock_quantity, image_path
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON CONFLICT (sku) DO NOTHING
        """

        params = (
            str(row['Артикул']),
            row['Наименование'],
            row['Описание товара'],
            category_id,
            manufacturer_id,
            supplier_id,
            row['Ед.измерения'],
            float(row['Цена']),
            float(row['Действующая скидка']),
            int(row['Количество на складе']),
            row['Изображение']
        )

```

```

        if self.db.execute_insert(query, params):
            count += 1

    print(f" Товаров: {count}")
    return True

# Импорт пунктов выдачи из pickup_points.xlsx
def import_pickup_points(self, df_pickup):
    count = 0

    for _, row in df_pickup.iterrows():
        query = "INSERT INTO pickup_points (postal_code, address) VALUES (%s, %s)"

        if self.db.execute_insert(query, (row['Почтовый индекс'], row['Адрес'])):
            count += 1

    print(f" Пунктов выдачи: {count}")
    return True

# Импорт пользователей из users.xlsx
def import_users(self, df_users):
    count = 0

    for _, row in df_users.iterrows():
        query = """
            INSERT INTO users (login, password_hash, full_name, role)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (login) DO NOTHING
        """

        if self.db.execute_insert(query, (row['Логин'], row['Пароль'], row['ФИО'], row['Роль'])):
            count += 1

    print(f" Пользователей: {count}")
    return True

# Импорт заказов. Вставляет запись в orders и связанные order_items.
def import_orders(self, df_orders):
    count = 0

    for _, row in df_orders.iterrows():
        user_id = self.db.get_id_by_name('users', 'full_name', row['ФИО клиента'])
        pickup_id = row['Пункт выдачи (ID)']

        query_order = """
            INSERT INTO orders (user_id, status, pickup_point_id, created_at, delivery_date)
            VALUES (%s, 'placed', %s, %s, %s)
            RETURNING id
        """

        try:
            cur = self.db.conn.cursor()
            cur.execute(query_order, (user_id, pickup_id, row['Дата заказа'], row['Дата поставки']))
            order_id = cur.fetchone()[0]

            product_id = row['Артикул']

            query_product = "SELECT price, discount_percent FROM products WHERE id = %s"
            cur.execute(query_product, (product_id,))
            product_data = cur.fetchone()

            if product_data:
                query_item = """

```

```

        INSERT INTO order_items (
            order_id, product_id, quantity,
            price_at_moment, discount_percent_moment
        ) VALUES (%s, %s, 1, %s, %s)
    """
    cur.execute(query_item, (order_id, product_id, product_data[0], product_data[1]))
    count += 1

    self.db.conn.commit()
    cur.close()
except Exception as e:
    self.db.conn.rollback()
    print(f"Ошибка импорта заказа: {e}")

print(f" Заказов: {count}")
return True

# Основной запуск последовательности импорта
def run(self):
    print("="*60)
    print(f"ИМПОРТ ДАННЫХ ИЗ ПАПКИ: {self.folder_name}")
    print("="*60)

    print(f"\nПолный путь: {self.base_path.absolute()}")

    if not self.base_path.exists():
        print(f"\nОШИБКА: Папка не существует")
        return False

    print("\n1. Чтение Excel файлов...")

    try:
        df_products = pd.read_excel(self.base_path / 'products.xlsx')
        print(f" products.xlsx: {len(df_products)} строк")

        df_pickup = pd.read_excel(self.base_path / 'pickup_points.xlsx')
        print(f" pickup_points.xlsx: {len(df_pickup)} строк")

        df_users = pd.read_excel(self.base_path / 'users.xlsx')
        print(f" users.xlsx: {len(df_users)} строк")

        df_orders = pd.read_excel(self.base_path / 'orders.xlsx')
        print(f" orders.xlsx: {len(df_orders)} строк")
    except FileNotFoundError as e:
        print(f"\nОШИБКА: Файл не найден - {e}")
        return False
    except Exception as e:
        print(f"\nОШИБКА чтения файлов: {e}")
        return False

    print("\n2. Импорт справочников...")
    self.import_categories(df_products)
    self.import_manufacturers(df_products)
    self.import_suppliers(df_products)

    print("\n3. Импорт товаров...")
    self.import_products(df_products)

    print("\n4. Импорт пунктов выдачи...")
    self.import_pickup_points(df_pickup)

    print("\n5. Импорт пользователей...")
    self.import_users(df_users)

```

```

        print("\n6. Импорт заказов...")
        self.import_orders(df_orders)

        print("\n" + "="*60)
        print("ИМПОРТ ЗАВЕРШЕН УСПЕШНО")
        print("="*60)
        return True

# Так как мы запускаем import_module.py напрямую
if __name__ == "__main__":

    # Фиксированное имя поддиректории, укажите здесь свой вариант:
    folder_name = "01_ГаджетМаркет"
    print(f"\nБудет использована фиксированная директория: imports/{folder_name}")

    print("\nПодключение к базе данных...")
    db_password = input("Введите пароль PostgreSQL: ").strip()

    if not db_password:
        print("ОШИБКА: Пароль не может быть пустым")
        exit()

    db = Database(password=db_password)

    if not db.connect():
        print("\nОШИБКА: Не удалось подключиться к БД")
        exit()

    print("Соединение установлено\n")

    try:
        importer = DataImporter(db, folder_name)
        success = importer.run()
    except Exception as e:
        print(f"\nОШИБКА выполнения импорта: {e}")
        success = False
    finally:
        db.close()

    if success:
        print("\nДанные успешно импортированы")
    else:
        print("\nИмпорт завершился с ошибками")

```

Сохраните import_module.py. Если библиотека pandas не установлена – установите ее через терминал VSCode выполнив команду – **pip install pandas**.

Убедитесь, что данные вариантов находятся в C:\shop_app\imports и запустите скрипт импорта (import_module.py). Убедитесь, что импорт завершился без ошибок и данные присутствуют в БД, для этого:

1. Запустите pgAdmin4 (введите пароль если потребуется)
2. Откройте БД shop_db
3. Откройте Schemas, выберите там Tables
4. Щелкните правой клавишей мыши на Tables – в контекстном меню выберите Query Tool – откроется окно запросов SQL.
5. Введите запрос: SELECT * from users

6. Нажмите F5 или кнопку запуска на панели инструментов Query Tool, что выполнить запрос

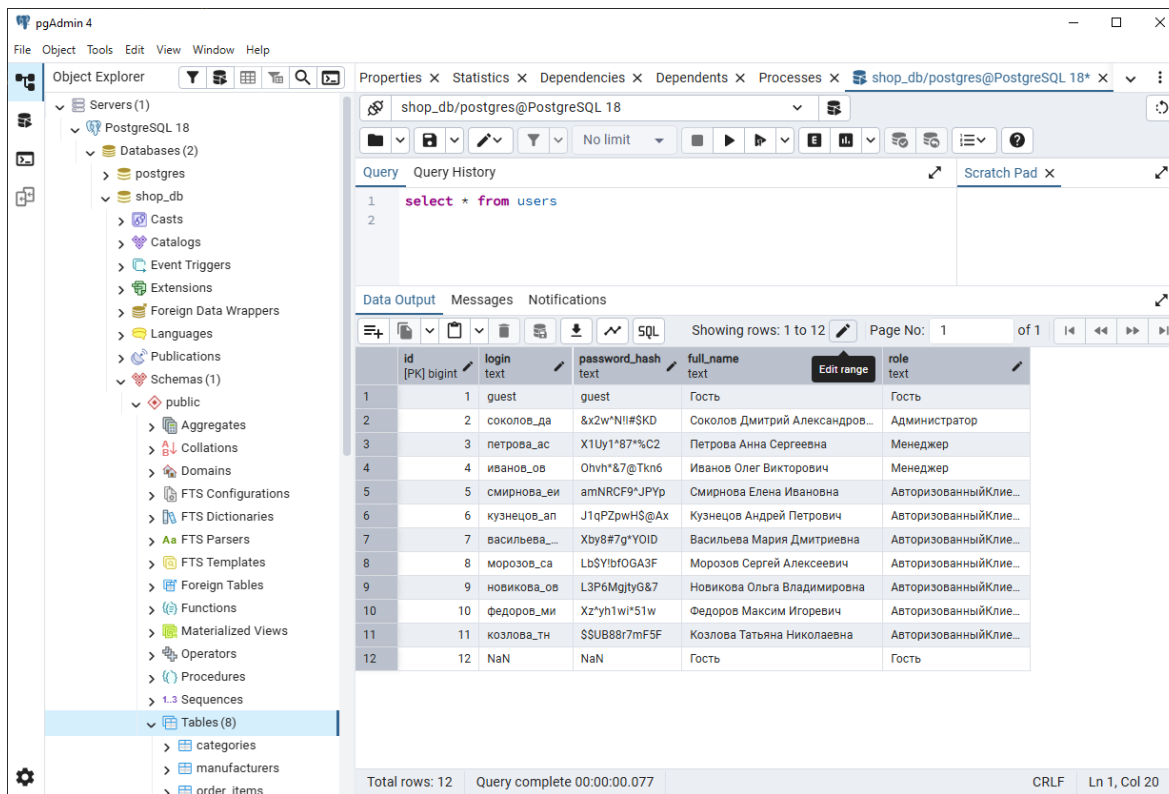


Рисунок 6.1 – Результат запроса, данные импортированы

Повторите тоже самое для всех остальных таблиц – убедитесь, что данные из файлов занесены в БД.